FRED TALKS

26th July 2026

CONTENTS

Zen of AI Coding

YOAV AVIRAM · 08 MAR 2026 · SOURCE

Dedicated to all those who are sceptical about the significance of agentic coding, and to those who are not, and are wondering what it means for the future of their profession. The title is an homage to Zen of Python by Tim Peters. Unlike Tim, I am not a zen master. My only aim is to take stock of where we are and where we might be heading. I have been building with coding agents daily for the past year, and I also help teams adopt them without losing reliability or security.

1. Software development is dead
2. Code is cheap
3. Refactoring easy
4. So is repaying technical debt
5. All bugs are shallow
6. Create tight feedback loops
7. Any stack is *your* stack
8. Agents are not just for coding
9. The context bottleneck is in your head
10. Build for a changing world
11. When considering Buy vs. Build, the answer, more often, is build
12. Fast rubbish is still rubbish
13. Software is a liability, a product is an asset
14. Moats are more expensive
15. Build for agents

16. Anticipate modes of failure

Software development is dead

You do not need to write another line of code if you do not want to. Coding agents can accomplish most coding tasks with the right direction.

Software development, as the act of manually producing code, is dying. A new discipline is being born. Your role in it is vital, but it is no longer centered on typing code. It is centered on framing problems, shaping context, defining constraints, and judging outcomes.

The marginal cost of code is collapsing. That single fact changes everything that follows.

Code is cheap

The economics of software have changed.

When coding is cheap, implementation stops being the constraint. You can build ten things in parallel. You cannot decide, validate, and ship ten things in parallel, at least not without changing the rest of the pipeline.

Cost of delay shifts. It is no longer about developer days. It is about time stuck in other bottlenecks: product decisions, unclear requirements, security review, user testing, release processes, and operational risk. Agents can flood these queues. Inventory grows. Lead time grows. Delay becomes more expensive, not less.

This changes prioritization. The highest leverage work is what unblocks shipping: tight feedback loops, tests, evals, guardrails, observability, and clear acceptance criteria. Anything that turns “we can build it” into “we can trust it”.

Agents can help alleviate some of these bottlenecks. The challenge is applying them outside of coding (see Agents are not just for coding)

Refactoring is easy

The cost of changing your mind is lower than it has ever been. Architectural decisions that once felt permanent are now provisional.

You chose React. Two months later, you regret it. Ask an agent to rewrite the project.

Making imperfect decisions is no longer fatal. In fact, it can be productive. A flawed reference implementation provides better context than a pristine specification. Agents reason more effectively from concrete artifacts than from abstract intent.

Rapid iteration is now the default mode. Over the last three months, we rebuilt our CMS four times from scratch, each with a different architecture. Each time we learned more about what we actually needed and what works.

When code is cheap, you can run more small bets.

So is repaying technical debt

There is little excuse for stale dependencies or ignored security patches. Updating libraries, migrating APIs, modernizing patterns. These are prompts away.

Prune your code regularly. Upgrade aggressively. Keep the surface clean.

Technical debt has not disappeared. But the cost of servicing it has dropped. That changes the incentive structure. Neglect becomes less defensible.

All bugs are shallow

Linus's law states that "given enough eyeballs, all bugs are shallow." Those eyeballs are now abundant.

Ask one model to review your code. Ask another. They may find different issues. Ask them to fix what they find. Often they will succeed.

How "dense" are bugs given a piece of code is a philosophical question. Bugs are not magically gone. Agents do not achieve perfection. From a practical perspective, however, the limiting factor is no longer the number of reviewers. It is the tightness of your feedback loops.

The claim here is profound: comprehension of the codebase at the function level is no longer necessary. At the same time, relying entirely on the models is a recipe for disaster. You still have a job! (see: create tight feedback loops, anticipate modes of failure)

Agents can sometimes one-shot a task.

We've built agents that create a complete custom-made marketing website in one shot, but this is by far the exception. In most cases, agents operate best when iterating towards a well-defined goal, guided by feedback.

Good tests are the first feedback loop. The agent will iterate until the tests pass. It is your responsibility to make sure the tests are adequate (by instructing the agent to create good ones) and to make sure the agent does not cheat (by weakening the tests or simply skipping them).

Give your agent access to your CI to create a second feedback loop. Give it access to your server logs to create a third. Giving the agent away to visually inspect a UI is yet another example.

Your job is to design environments where iteration converges toward correctness instead of drifting toward plausible nonsense.

Velocity without feedback is chaos.

Any stack is your stack

Agents are competent across most major stacks where training data exists. Since you are not writing the code, your attachment to a specific stack becomes less relevant.

Do not limit yourself to what you personally know. Your conceptual understanding transfers more than you think.

When you lack terminology or domain knowledge, ask the agent to brief you. The barrier to entry into unfamiliar ecosystems is lower than it has ever been.

Agents are not just for coding

Coding is only the beginning.

Agents can assist with business analysis, UX, infrastructure, operations, ad campaigns, analytics configuration, even accounting workflows.

I migrated four WordPress blogs from an overpriced host to Hetzner in minutes after postponing the task for years. The blocker was never a technical difficulty. It was attention. Claude completed the task in 15 minutes.

Agents reduce the activation energy required to execute tedious but valuable work.

Grant access carefully. Limit scope. Audit results. Direction without oversight is reckless.

The context bottleneck is in your head

Context engineering has improved. Tooling has improved.

Yet when running multiple agents in parallel, the real bottleneck is human coordination.

You must track what each agent is doing. You must remember which assumptions were made. You know what questions to ask next.

As agents become better at managing their own context, your ability to supervise them becomes the limiting factor.

The constraint is no longer tokens. It is cognition.

Build for a changing world

New models. New capabilities. New frameworks. New skills.

The ground shifts daily. Some models are better at reasoning. Some at coding. Some at translation.

Flexibility is not optional. It must be built into your workflow and products. Both must be engineered for experimentation and adaptability.

When considering Buy vs. Build, the answer, more often, is build

When code was expensive, buying made sense. When code approaches zero marginal cost, the calculus changes.

Every paid API is now a question. Could we replicate this internally? Should we?

For our QA agent, we needed full-page screenshots of live sites. Many APIs offered this. Instead, we deployed Playwright to AWS Lambda, tailored to our needs, and hardened.

This is not a blanket argument against SaaS. Maintenance, security, and scale still matter. But many small and medium services that once required vendors are now within reach.

Some call this the end of SaaS.

Fast rubbish is still rubbish

Speed is intoxicating. It is also dangerous.

You can generate enormous amounts of code quickly. But velocity and lines of code written were always terrible indicators of progress.

Refactoring is cheap, but it is cheaper to detect flawed direction early.

Discipline matters more, not less, when execution is frictionless.

Software is a liability, a product is an asset

Software costs money to maintain. It only becomes an asset when it produces value for someone.

This was always true. It is harder to remember now because building is so easy.

It is tricker then ever to resist the temptation to add features. Resist it. Build what is used. Kill what is not.

Feature gaps close quickly. What once took years to replicate can now take months. What took months can now take days.

A feature set is no longer a durable moat.

Moats shift toward distribution, data, brand, network effects, regulation, and ecosystem integration.

The barrier to entry falls. The barrier to defensibility rises.

There is a new hierarchy.

At the base are models (Opus 4.6, GPT-5.3-codex). On top of them sit agents (Claude Code, OpenClaw). On top of agents sit services.

Today, agents use services designed for humans. It works, but it is inefficient. The next step is services designed for agents (Moltbook being the harbinger).

Make your services accessible to agents. Expose structured context. Provide machine-readable surfaces.

Agentic commerce is emerging. In some cases, a markdown endpoint is enough. In others, the service itself must be designed agent-first.

Agent-first is not a tooling choice. It is a product decision. It changes your interfaces, your reliability model, your metrics, and what you build next.

We are building an agent-first CMS. If we succeed, humans will not need to use it, though they can.

AX is the new UX

Do not assume competence implies perfection.

Like an engineer overseeing the construction of a bridge, your job is not to lay bricks. It is to ensure the structure does not collapse.

Agents will make mistakes. They will drift off course. They will introduce subtle flaws. They will create security vulnerabilities. One of mine attempted to commit a .env file to git. They can be inventive in how they self-sabotage.

Be one step ahead. Ask yourself how this could break. Where could it leak. What could it expose. What assumptions is the agent quietly making.

Play the what-if game relentlessly. Then build guardrails.

Automate checks. Lock down permissions. Isolate environments. Add monitoring. Create recovery paths. Make rollback trivial.

Remember, code is cheap, and you have new superpowers. Build tools to allow you to probe deep, discover vulnerabilities, and test for anticipated failure modes. Design systems that expect failure and degrade gracefully.

This is the distinction between software development and software engineering.

I work with engineering and product teams adopting agents. The goal is simple: ship faster without lowering standards.

I usually help in two ways:

Agent-first software engineering
We turn agents into part of the system, not a novelty. We design workflows, guardrails, and supporting tools so the team can delegate confidently.

If you want to operationalize coding agents without turning your codebase into a casino, get in touch.

Agent-first product strategy
I help companies shift products from human-centric interfaces to agent-accessible, and where it makes sense, agents-first. That means clarifying what agents should be able to do, what constraints they must operate under, and what needs to change in your APIs, permissions, reliability model, and product surface area to support them.

This typically results in an agent-access roadmap, an operating model, and a prioritized set of product changes that make agents a first-class user.

I'm the co-founder of WhiteBoar, where AI agents build your brand and launch visually striking marketing websites, complete with professional multi-lingual content, in minutes, not months. Contact me at yoav@whiteboar.it.

How Claude Code navigates large codebases

CLAUDE · 18 MAY 2026 · SOURCE

Claude Code is running in production across multi-million-line monorepos, decades-old legacy systems, distributed architectures spanning dozens of repositories, and at organizations with thousands of developers. These environments present challenges that smaller, simpler codebases don't, whether that's build commands that differ across every subdirectory or legacy code spread across folders with no shared root.

This article covers the patterns we've observed that have led to successful adoption of Claude Code at scale. We use "large codebase" to refer to a wide range of deployments: monorepos with millions of lines, legacy systems built over decades, dozens of microservices across separate repositories, or any combination of the above. That also includes codebases running on languages that teams don't always associate with AI coding tools, such as C, C++, C#, Java, PHP. (Claude Code performs better than most teams expect it to in those cases, particularly as of recent model releases.) While every large codebase deployment is shaped by its specific version control, team structure, and accumulated conventions, the patterns here generalize across them and are a good starting point for teams considering adopting Claude Code.

Claude Code navigates a codebase the way a software engineer would: it traverses the file system, reads files, uses grep to find exactly what it needs, and follows references across the codebase. It operates locally on the developer's machine and doesn't require a codebase index to be built, maintained, or uploaded to a server.

RAG-powered AI coding tools work by embedding the entire codebase and retrieving relevant chunks at query time. At large scale, those systems can fail because embedding pipelines can't keep up with active engineering teams. By the time a developer queries the index, it reflects the codebase as it previously existed weeks, days, or even hours before. Retrieval then returns a function the team renamed two weeks ago, or references a module that was deleted in the last sprint, with no indication that either is out of date.

Agentic search avoids those failure modes. There's no embedding pipeline or centralized index to maintain as thousands of engineers commit new code. Each developer's instance works from the live codebase.

But the approach has a tradeoff: it works best when Claude has enough starting context to know where to look. This means the quality of Claude's navigation is shaped by how well the codebase is set up, layering context with CLAUDE.md files and skills. If you ask it to find all instances of a vague pattern across a billion-line codebase, you'll hit context-window limits before the work begins. Teams that invest in codebase setup see better results.

The harness matters as much as the model

One of the most common misconceptions about Claude Code is that its capabilities are solely defined by the model used. Teams focus on a model's benchmarks and how it performs on test tasks. In practice, the ecosystem built around the model—the harness—determines how Claude Code performs more than the model alone.

The harness is built from five extension points—CLAUDE.md files, hooks, skills, plugins, and MCP servers—each serving a different function. The order in which teams build them matters, as each layer builds on what came before. Two additional capabilities, LSP integrations and subagents, round out the setup. Below, we explain what each of these components and capabilities do:

CLAUDE.md files come first. These are context files that Claude reads automatically at the start of every session: root file for the big picture, subdirectory files for local conventions. They give Claude the codebase knowledge it needs to do anything well. Because they load in every session regardless of the task, keeping them focused on what applies broadly will prevent them from becoming a drag on performance.

Hooks make the setup self-improving. Most teams think of hooks as scripts that prevent Claude from doing something wrong, but their more valuable use is continuous improvement. A stop hook can reflect on what happened during a session and propose CLAUDE.md updates while the context is fresh. A start hook can load team-specific context dynamically so every

developer gets the right setup for their module without manual configuration. For automated checks like linting and formatting, hooks enforce the rules deterministically and produce more consistent results than relying on Claude to remember an instruction.

Skills keep the right expertise available on-demand without bloating every session. In a large codebase with dozens of task types, not all expertise needs to be present in every session. Skills solve this through progressive disclosure, offloading specialized workflows and domain knowledge that would otherwise compete for context space and loading them only when the task calls for it. For example, a security review skill loads when Claude is assessing code for vulnerabilities, while a document processing skill loads when a code change is made and documentation needs to be updated.

Skills can also be scoped to specific paths so they only activate in the relevant part of the codebase. A team that owns a payments service can bind their deployment skill to that directory, so it never auto-loads when someone is working elsewhere in the monorepo.

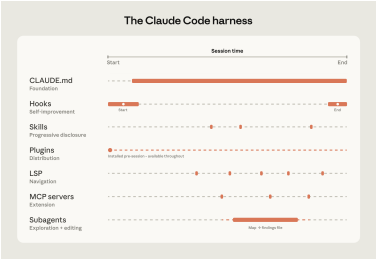
Plugins distribute what works. One challenge with large codebases is that *good* setups can stay tribal. A plugin bundles skills, hooks, and MCP configurations into a single installable package, so when a new engineer installs that plugin on day one, they will immediately have the same context and capabilities as those who have been using Claude already. Plugin updates can be distributed across the organization through managed marketplaces.

For example, a large retail organization we work with built a skill connecting Claude to their internal analytics platform so that business analysts could pull performance data without leaving their workflow. They distributed it as a plugin before the broad rollout to the business.

Language server protocol (LSP) integrations give Claude the same navigation a developer has in their IDE. Most large-codebase IDEs already have an LSP running, powering "go to definition" and "find all references." Surfacing this to Claude gives it symbol-level precision: it can follow a function call to its definition, trace references across files, and distinguish between identically named functions in different languages. Without it, Claude pattern-matches on text and can land on the wrong symbol. One enterprise software company we worked with deployed LSP integrations org-wide before their Claude Code rollout, specifically to make C and C++ navigation reliable at scale. For multi-language codebases, this is one of the highest-value investments.

MCP servers extend everything. MCP servers are how Claude connects to internal tools, data sources, and APIs that it can't otherwise reach. The most sophisticated teams built MCP servers exposing structured search as a tool Claude can call directly. Others connect Claude to internal documentation, ticketing systems, or analytics platforms.

Subagents split exploration from editing. A subagent is an isolated Claude instance with its own context window that takes a task, does the work, and returns only the final result to the parent. Once the harness is in place, some teams spin up a read-only subagent to map a subsystem and write findings to a file, then have the main agent edit with the full picture.



Claude Code's extension layer at a glance.

The table below summarizes what each component does, when it loads, and the most common mistakes we see with each:

Component	What it is	When it loads	Best for	Common confusion
CLAUDE.md	Context file Claude reads automatically	Every session	Project-specific conventions, codebase knowledge	Using it for reusable expertise that belongs in a skill
Hooks	Scripts that run at key moments	Triggered by events	Automating consistent behavior, capturing session learnings	Using prompts for things that should run automatically
Skills	Packaged instructions for specific task types	On demand, when relevant	Reusable expertise across sessions and projects	Loading everything into CLAUDE.md instead
Plugins	Bundled skills, hooks, MCP configs	Always available once configured	Distributing a working setup across the org	Letting good setups stay tribal
Language server protocol (LSP)*	Real-time code intelligence via language specific servers	Always available once configured	Symbol-level navigation and automatic error detection in typed languages	Assuming that it's automatic
MCP servers	Connections to external tools and data	Always available once configured	Giving Claude access to internal tools it can't otherwise reach	Building MCP connections before the basics are working

Subagents*	Separate Claude instances for specific tasks	When invoked	Splitting exploration from editing, parallel work	Running exploration and editing in the same session

*LSP is accessed through the plugin layer. Subagents are a delegation capability rather than a configured extension point.

Three configuration patterns from successful deployments

How you configure Claude Code for a large codebase depends heavily on how that codebase is structured. Still, three patterns appeared consistently across the deployments we observed.

Making the codebase navigable at scale

Claude's ability to help in a large codebase is bounded by its ability to find the right context. Too much context loaded into every session degrades performance, while too little context leaves Claude to navigate blind. The most effective deployments invest upfront in making the codebase legible to Claude. A few patterns appear consistently:

- **Keeping CLAUDE.md files lean and layered.** Claude loads them additively as it moves through the codebase: root file for the big picture, subdirectory files for local conventions. The root file should be pointers and critical gotchas only; everything else drifts into noise.

- **Initializing in subdirectories, not at the repo root.** Claude works best when it's scoped to the part of the codebase that's actually relevant to the task. In monorepos, this can feel counterintuitive because tooling often assumes root access, but Claude automatically walks up the directory tree and loads every CLAUDE.md file it finds along the way, so root-level context is never lost.

- **Scoping test and lint commands per subdirectory.** Running the full suite when Claude changed one service causes timeouts and wastes context on irrelevant output. CLAUDE.md files at the subdirectory level should specify the commands that apply to that part of the codebase. This works well for service-oriented codebases where each directory has its own test and build commands. In compiled-language monorepos with deep cross-directory dependencies, per-subdirectory scoping is harder to achieve and may require project-specific build configurations.

- **Using `.ignore` files to exclude generated files, build artifacts, and third-party code.** Committing `permissions.deny` rules in `.claude/settings.json` means the exclusions are version-controlled, so every developer on the team gets the same noise reduction without configuring it themselves. In some codebases, generated files are themselves the subject of development work. Developers who work on code generators can override project-level exclusions in their local settings without affecting the rest of the team.

- **Building codebase maps when the directory structure doesn't do the work.** For organizations where code isn't consolidated in a conventional directory structure, a lightweight markdown file at the repo root listing each top-level folder with a one-line description of what lives there gives Claude a table of contents it can scan before opening files. For codebases with hundreds of top-level folders, this works best as a layered approach: the root file describes only the highest-level structure, and subdirectory CLAUDE.md files provide the next level of detail, loading on demand as Claude moves through the tree. For simpler cases, @-mentioning the specific files or directories Claude should reference can do the same job.

- **Running LSP servers so Claude searches by symbol, not by string.** Grep for a common function name in a large codebase returns thousands of matches and Claude burns context opening files to figure out which matters. LSP returns only the references that point to the same symbol, so the filtering happens before Claude reads anything. Setting this up requires installing a code intelligence plugin for your language and the corresponding language server binary; the Claude Code documentation covers the available plugins and troubleshooting.

One caveat: there are edge cases where even the hierarchical CLAUDE.md approach breaks down, for example codebases with hundreds of thousands of folders and millions of files, or legacy systems on non-git version control. We will address their challenges in future installments of this series.

Actively maintaining CLAUDE.md files as model intelligence evolves

As models evolve, instructions written for your current model can work against a future one. CLAUDE.md files that guided Claude through patterns it used to struggle with may either become unnecessary or actively constraining when the next model ships. For example, a CLAUDE.md rule that tells Claude to break every refactor into single-file changes may have helped an earlier model stay on track but would prevent a newer one from making coordinated cross-file edits it handles well.

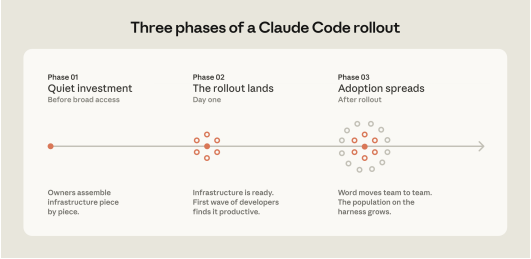
Skills and hooks built to compensate for specific model limitations, whether in the model's reasoning or in Claude Code's own tooling, become overhead once those limitations no longer exist. A hook that intercepted file writes to enforce p4 edit in a Perforce codebase, for example, became redundant once Claude Code added native Perforce mode.

Teams should expect to do a meaningful configuration review every three to six months, but it's also worth doing one whenever performance feels like it's plateaued after major model releases.

Assigning ownership for Claude Code management and adoption

Technical configuration alone doesn't drive adoption. The organizations that got it right invested in the organizational layer, too.

The rollouts that spread fastest had a dedicated infrastructure investment before broad access. A small team, sometimes even just one person, wired up the tooling so Claude already fit developer workflows when they first touched it. At one company, a couple of engineers built a suite of plugins and MCPs that were available on day one. At another, an entire team focused on managing AI coding tools had the infrastructure in place before the rollout began. In both cases, developers' first experience was productive rather than frustrating, and adoption spread from there.



The teams doing this work today tend to sit under developer experience or developer productivity, which is typically the function responsible for onboarding new engineers and building developer tooling. An emerging role in several organizations is an agent manager: a hybrid PM/engineer function dedicated to managing the Claude Code ecosystem. For organizations without a dedicated team, the minimum viable version is a DRI: one person with ownership over the Claude Code configuration, the authority to make calls on settings, permissions policy, the plugin marketplace, and CLAUDE.md conventions, and the responsibility to keep them current.

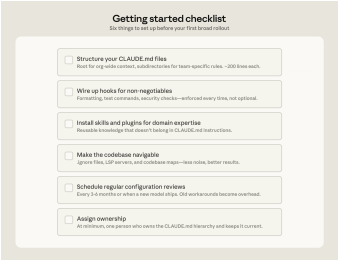
Bottoms-up adoption generates enthusiasm but can fragment without someone to centralize what works. You need to have an individual or a team assemble and evangelize the right Claude Code conventions (such as a standardized CLAUDE.md hierarchy or a curated set of skills and plugins). Without that work, knowledge will stay tribal and adoption will plateau.

In large organizations, especially those in regulated industries, governance questions come up early, such as: who controls which skills and plugins are available, how do you prevent thousands of engineers from independently rebuilding the same thing, how do you make sure AI-generated code goes through the same review process as human-generated code? To address these early on, we suggest starting with a defined set of approved skills, required code review processes, and limited initial access, and expand as confidence builds.

We've observed the smoothest deployments at organizations that establish cross-functional working groups early by bringing together engineering, information security, and governance representatives to define requirements together and build a rollout roadmap.

Applying these patterns to your organization

Claude Code is designed around conventional software engineering environments where engineers are the primary codebase contributors, the repo uses Git, and code follows standard directory structures. Most large codebases fit this mold, but non-traditional setups such as game engines with large binary assets, environments with unconventional version control, or non-engineers contributing to the codebase require additional configuration work. Our guidance assumes a conventional setup and the patterns we've described have worked across many of our customers. Any remaining complexity requires judgment specific to your codebase, tooling, and organization. That's where Anthropic's Applied AI team works directly with engineering teams to translate these patterns into your organization's specific requirements.



Acknowledgements: *Special thanks to Alon Krifcher, Charmaine Lee, Chris Concannon, Harsh Patel, Henrique Savelli, Jason Schwartz, Jonah Dueck and Kirby Kohlmorgen from Anthropic's Applied AI team for sharing their experience deploying Claude Code at scale, and to Amit Navindgi at Zoox for providing feedback on this article.*

[Daily article] July 26: Nile

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 26 JUL 2026 · SOURCE

The Nile is a major river in northeast Africa that empties into the Mediterranean Sea. At 7,088 kilometers (4,404 mi) in length, it is the longest river in the world, although the volume of water it carries is much smaller than other major rivers. The Nile has two major tributaries: the White Nile and the Blue Nile. Geologically, the Nile is a young river and has followed its present course for only 15,000 years. The Nile was the foundation of the ancient Egyptian civilization, which relied on the river for nearly every aspect of life. Many Europeans were fascinated by the Nile, and their explorations around Lake Victoria in the late 19th century located the source of the river. The Nile plays a critical role in the economies of Egypt and Sudan, which rely on it for irrigation. More than a dozen dams have been built in the Nile Basin, altering the river's annual flood cycle and restricting the transport of silt downstream to the Nile Delta. (Full article...).

Read more: https://en.wikipedia.org/wiki/Nile

Today's selected anniversaries:

1882:

Boer mercenaries established the Republic of Stellaland (later flag pictured) in land claimed by the United Kingdom as part of British Bechuanaland. https://en.wikipedia.org/wiki/Stellaland

1953:

The Battle of the Samichon River, the last engagement of the Korean War, ended a few hours before the signing of the Korean Armistice Agreement. https://en.wikipedia.org/wiki/Battle_of_the_Samichon_River

1968:

After coming second to Nguyễn Văn Thiệu in a rigged presidential election, Trương Đình Dzu was jailed by a South Vietnamese military court for illicit currency transactions. https://en.wikipedia.org/wiki/Tr%C6%B0%C6%A1ng_%C4%90%C3%ACnh_Dzu

2012:

The New Irish Republican Army was formed from a merger of a number of dissident republican militant groups. https://en.wikipedia.org/wiki/New_Irish_Republican_Army

---

Wiktionary's word of the day:

first strike: 1. (military) An attack (typically nuclear) launched without warning in the absence of a prior attack by one's opponent, with the intention of destroying one's opponent's ability to retaliate. 2. (criminal law) A first offense by a lawbreaker, as counted by a three-strikes law. 3. (numismatics) One of the first few coins struck from a new set of dies. 4. About Word of the Day 5. Nominate a word 6. Leave feedback https://en.wiktionary.org/wiki/first_strike

---

Wikiquote quote of the day:

The pendulum of the mind oscillates between sense and nonsense, not between right and wrong. The numinosum is dangerous because it lures men to extremes, so that a modest truth is regarded as the truth and a minor mistake is equated with fatal error. --Carl Jung https://en.wikiquote.org/wiki/Carl_Jung